28 Data Management

By Diana C. Bouchard

Topic Highlights

Data Relationships, Storage and Retrieval, Quality Issues
Database Structure, Types, Operation, Software and Maintenance
Basics of Database Design
Queries and Reports
Special Requirements of Real-Time Process Databases
Data Documentation and Security

28.1 Introduction

Data are the lifeblood of industrial process operations. The levels of efficiency, quality, flexibility, and cost reductions needed in today's competitive environment cannot be achieved without a continuous flow of accurate, reliable information. Good data management ensures the right information is available at the right time to answer the needs of the organization. Databases store this information in a structured repository and provide for easy retrieval and presentation in various formats.

28.2 Database Structure

The basic structure of a typical database consists of *records* and *fields*. A *field* contains a specific type of information—for example, the readings from a particular instrument or the values of a particular laboratory test. A *record* contains a set of related field values, typically taken at one time or associated with one location in the plant. In a spreadsheet, the fields would usually be the columns (variables) and the records would be the rows (sets of readings).

In order to keep track of the information in the database as it is manipulated in various ways, it is desirable to choose a *key field* to identify each record, much as it is useful for people to have names so we can address them. Figure 28-1 shows the structure of a portion of a typical process database, with the date and time stamp as the key field.

28.3 Data Relationships

Databases describe relationships among *entities*, which can be just about anything: people, products, machines, measurements, payments, shipments, and so forth. The simplest kind of data relationship is *one-to-one*, meaning that any one of entity *a* is associated with one and only one of entity *b*. An example would be customer name and business address.

In some cases, however, entities have a *one-to-many* relationship. A given customer has probably made multiple purchases from your company, so customer name and purchase order number would have a

DateTime	Impeller Speed rpm	Additive Flowrate, L/min	Additive Concentration ppm	…
2005-05-20 02:00	70.1	24.0	545	…
2005-05-20 03:00	70.5	25.5	520	…
2005-05-20 04:00	71.1	25.8	495	…
2005-05-20 05:00	69.5	23.9	560	…
2005-05-20 06:00	69.8	24.2	552	…
…	…	…	…	…

Figure 28-1: Process Database Structure

one-to-many relationship. In other cases, *many-to-many* relationships exist. A supplier may provide you with multiple products, and a given product may be obtained from multiple suppliers.

Database designers frequently use *entity-relationship diagrams* (Figure 28-2) to illustrate linkages among data entities.

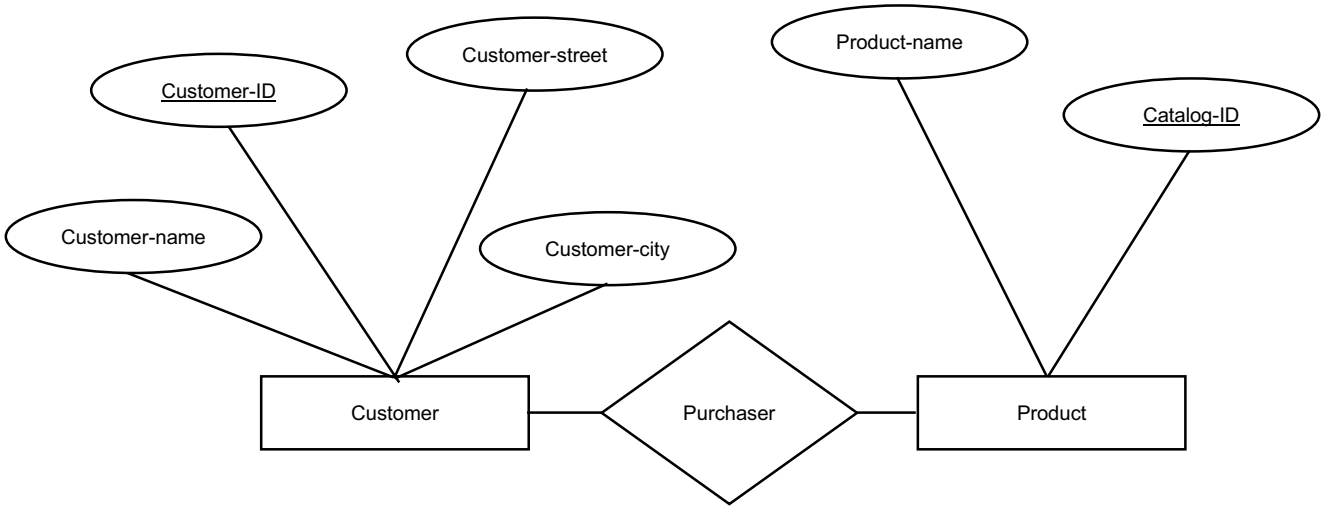


Figure 28-2: Typical Entity-Relationship Diagram

28.4 Database Types

The simplest database type is called a *flat file*, which is an electronic analogue of a file drawer, with one record per folder, and no internal structure beyond the two-dimensional (row and column) tabular structure of a spreadsheet. Flat-file databases are adequate for many small applications of low complexity.

However, if the data contain one-to-many or many-to-many relationships, the flat file structure cannot adequately represent these linkages. The temptation is to reproduce information in multiple locations, wherever it is needed. However, if you do this, and you need to update the information afterwards, it is easy to do so in some places and forget to do it in others. Then, your databases contain inconsistent and inaccurate information, leading to problems such as out-of-stock situations, wrong customer contact information, and obsolete product descriptions.

A better solution is to use a *relational database*. The essential concept of a relational database is that ALL information is stored as tables, both the data themselves and the relations between them. Each table

contains a key field which is used to link it with other tables. Figure 28-3 illustrates a relational database containing data on customers, products and orders for a particular industrial plant.

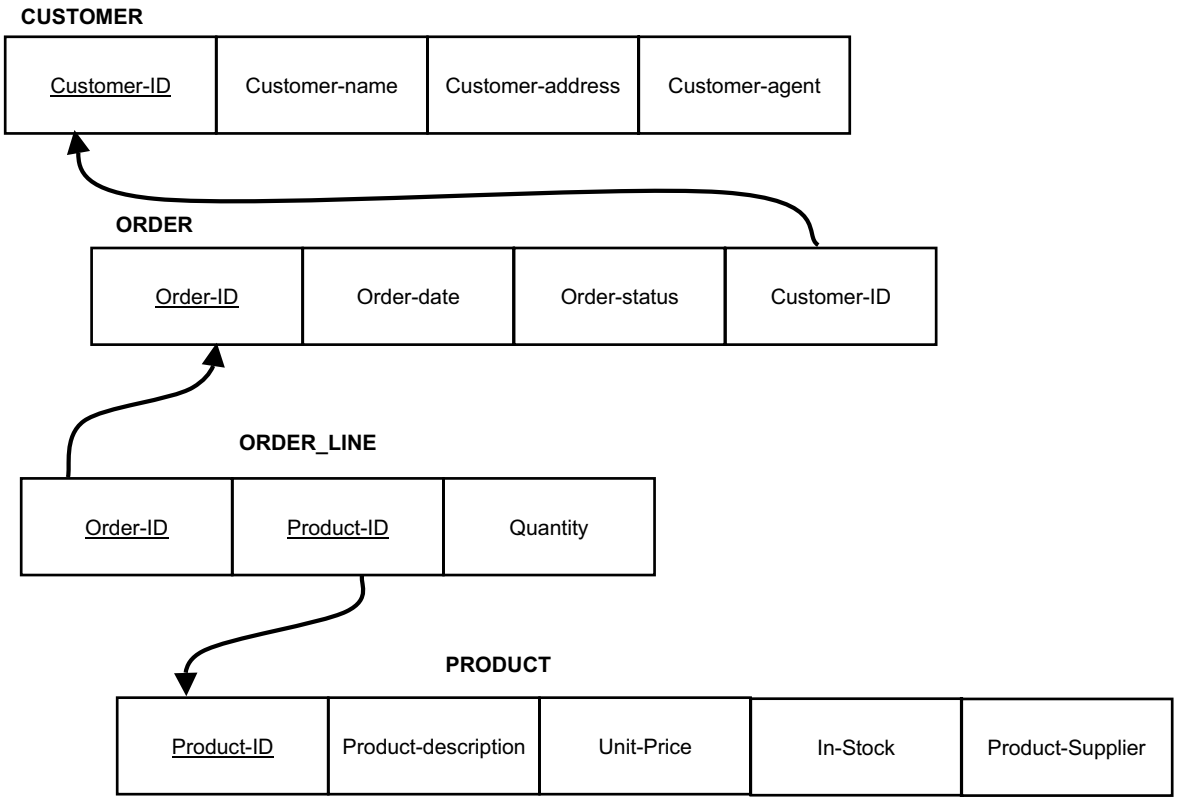


Figure 28-3: Relational Database Structure

Additional specifications describe how the tables in a relational database should be structured so the database will be reliable in use and robust against data corruption. The degree of conformity of a database to these specifications is described in terms of degrees of *normal form.*

28.5 Basics of Database Design

The fundamental principle of good database design is to create a database that will support the desired uses of the information it contains. Factors such as database size, volatility (frequency of changes), type of interaction desired with the database, and the knowledge and experience of database users will influence the final design.

Key fields must be unique to each record. If two records end up with the same key value, the likely result is misdirected searches and loss of access to valuable information.

Definition of the other fields is also important. Anything you might want to search or sort on should be kept in its own field. For example, if you put first name and last name together in a personnel database, you can never sort by last name.

28.6 Queries and Reports

A query is a request to a database to return information matching specified criteria. The criteria are usually stated as a logical expression using operators such as equal, greater than, less than, AND and OR. Only the records for which the criterion evaluates as TRUE are returned. Queries may be per-

formed via interactive screens, or using query languages such as SQL (Standard Query Language) which have been developed to aid in the formulation of complex queries and their storage for re-use (as well as more broadly for creating and maintaining databases). Figure 28-4 shows a typical SQL query.

```
SELECT PRODUCT_NAME, PRODUCT_CATEGORY,
PRODUCT_SERVICERATING, UNIT_PRICE
        FROM PRODUCT_FLOWMETER
        WHERE (PRODUCT_CATEGORY LIKE "%Coriolis"
        AND PRODUCT_SERVICERATING = "%Acid"
        AND UNIT_PRICE < 10000;
```

Figure 28-4: Typical SQL Query

Reports pull selected information out of a database and present it in a predefined format as desired by a particular group of end users. The formatting and data requirements of a particular report can be stored and used to regenerate the report as many times as desired using up-to-date data.

Interactive screens or a report definition language can be used to generate reports. Figure 28-5 illustrates a report generation screen.

28.7 Data Storage and Retrieval

How much disk storage a database requires depends on several factors: the number of records in the database, the number of fields in each record, the amount and type of information in each field, and how long information is retained in the database. Although computer mass storage has rapidly expanded in size and decreased in cost over the last few decades, human ingenuity in finding new uses for large quantities of data has steadily kept pace. Very large databases such as those used by retail giant Wal-Mart to track customer buying trends now occupy many terabytes (trillions of bytes) of storage space.

Managing such large databases poses a number of challenges. The simple act of querying a multi-terabyte database can become annoyingly slow. Important data relationships can be concealed by the sheer volume of data. As a response to these problems, data mining techniques have been developed to explore these large masses of information and retrieve information of interest. Assuring consistent and error-free data in a database which may experience millions of modifications per day is another problem.

Another set of challenges arises when two or more databases that were developed separately are interconnected or merged. For example, the merger of two companies often results in the need to combine their databases. Even within a single company, as awareness grows of the opportunities that can be seized by leveraging their data assets, management may undertake to integrate all the company's data into a vast and powerful data warehouse. Such integration projects are almost always long and costly, and the failure rate is high. But, when successful, they provide the company with a powerful data resource.

To reduce data storage needs, especially with process or other numerical data, data sampling, filtering and compression techniques are often used. If a reading is taken on a process variable every 10 min-

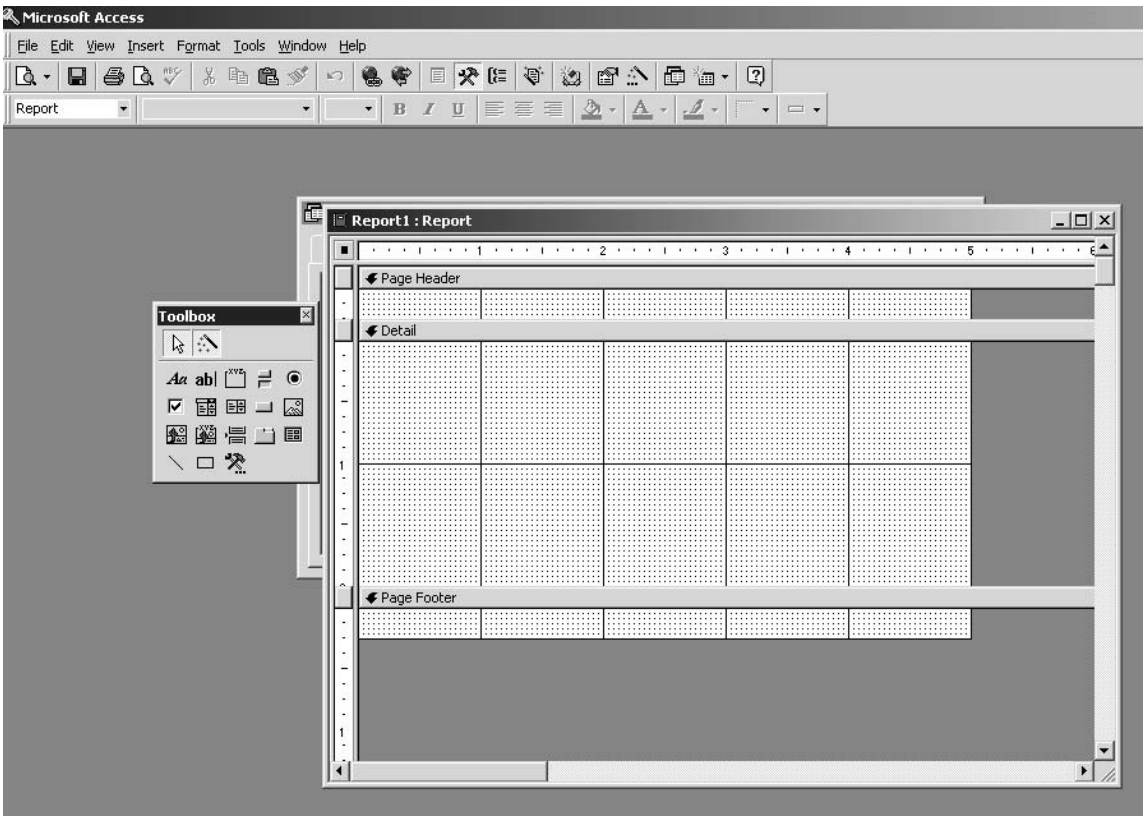


Figure 28-5: Report Generation Screen

utes as opposed to every minute, simple arithmetic will tell you that only 10% of the original data volume will need to be stored. However, a price is paid for this reduction: loss of any record of process variability on a timescale shorter than 10 minutes, and possible generation of spurious frequencies (aliasing) by certain data analytic methods. Data filtering is often used to eliminate certain values, or certain kinds of variability, that are judged to be noise. For example, values outside a predefined range, or changes occurring faster than a certain rate, may be removed.

Data compression algorithms define a band of variation around the most recent values of a variable and record a change in that variable only when its value moves outside the band (see Figure 28-6). Essentially the algorithm defines a "dead band" around the last few values and considers any change within that band to be insignificant. Once a new value is recorded, it is used to redefine the compression dead band, so it will follow longer-term trends in the variable. Detail variations in this family of techniques ensure a value is recorded from time to time even if no significant change is taking place, or adjust the width and sensitivity of the dead band during times of rapid change in variable values.

28.8 Database Operations

The classic way of operating on a database, such as a customer database in a purchasing department, is to maintain a master file containing all the information entered so far, and then periodically update the database using a transaction file containing new information. The key field in each transaction record is tested against the key field of each record in the master file to identify the record that needs to be modified. Then the new information from the transaction record is written into the master file, overwriting the old information (see Figure 28-7). This approach is well suited to situations where the information changes relatively slowly and the penalties for not having up-to-the-minute information are not severe. Transactions are typically run in batches one to several times a day.

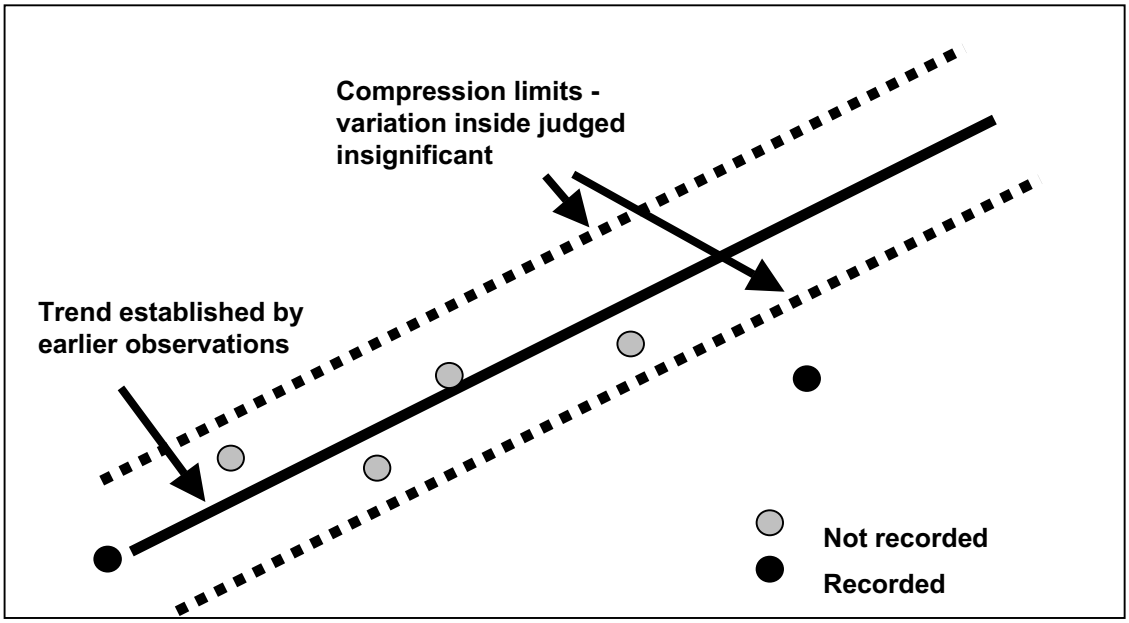


Figure 28-6: How a Data Compression Deadband Works

TRANSACTION FILE			
29177	company-29177	**agent-name-29177 (CHANGED)**	agent-phone-29177
30064	**company-30064**	**agent-name-30064**	**agent-phone-30064**
30195	company-30195	agent-name-30195	**agent-phone-30195 (CHANGED)**

New agent name - replace data

NEW RECORD - insert

New phone number - replace data

MASTER FILE			
28295	company-28295	agent-name-28295	agent-phone-28295
29003	company-29903	agent-name-29903	agent-phone-29903
29177	company-29177	agent-name-29177	agent-phone-29177
29804	company-29804	agent-name-29804	agent-phone-29804
30018	company-30018	agent-name-30018	agent-phone-30018
30122	company-30122	agent-name-30122	agent-phone-30122
30147	company-30147	agent-name-30147	agent-phone-30147
30195	company-30195	agent-name-30195	agent-phone-30195
31110	company-31110	agent-name-31110	agent-phone-31110

Figure 28-7: Interaction Between Transaction File and Master File

As available computer power increased and user interfaces improved, interactively updated databases became more common. In this case, a data entry worker types transactions into an on-screen form, directly modifying the underlying master file. Built-in range and consistency checks on each field min-

imize the chances of entering incorrect data. With the advent of fast, reliable computer networks and intelligent remote devices, transaction entries may come from other software packages, other computers, or portable electronic devices, often without human intervention. Databases can now be kept literally up-to-the-minute, as in airline reservation systems.

Since an update request can now arrive for any record at any moment (as opposed to the old batch environment where a computer administrator controlled when updates happened), the risk of two people or devices trying to update the same information at the same time has to be guarded against. File and record locking schemes were developed to block access to a file or record under modification, preventing other users from operating on it until the first user's changes were complete.

Other database operations include searching for records meeting certain criteria (e.g., with values for a certain variable greater than a threshold) or sorting the database (putting the records in a different order). Searching is done via queries, as already discussed. A sort can be in ascending order (e.g., A to Z) or descending order (Z to A). You can also do a sort within a sort (e.g., charge number within department) (see Figure 28-8).

A-Z Sort by Lastname

Lastname	PO Number
Anderson	38192844
Anderson	28691877
Anderson	31243896
Harris	31219925
LeMoyne	36645119
LeMoyne	30042894
Parrish	38456712
Williams	29943851

Ascending Sort by PO Number Within A-Z Sort by Lastname

Lastname	PO Number
Anderson	28691877
Anderson	31243896
Anderson	38192844
Harris	31219925
LeMoyne	30042894
LeMoyne	36645119
Parrish	38456712
Williams	29943851

Z-A Sort by Lastname

Lastname	PO Number
Williams	29943851
Parrish	38456712
LeMoyne	30042894
LeMoyne	36645119
Harris	31219925
Anderson	38192844
Anderson	31243896
Anderson	28691877

Ascending Sort by Purchase Order Number

Lastname	PO Number
Anderson	28691877
Williams	29943851
LeMoyne	30042894
Harris	31219925
Anderson	31243896
LeMoyne	36645119
Anderson	38192844
Parrish	38456712

Figure 28-8: Results of Different Sorting Operations

28.9 Special Requirements of Real-Time Process Databases

When the data source is a real-time industrial process, a number of new concerns arise. Every piece of data in a real-time process database is now associated with a timestamp and a location in the plant, and that information must be retained with the data. A real-time process reading also has an "expiry date" and applications that use that reading must verify that it is still good before using it. Data also come in many cases from measuring instruments which introduce concerns about accuracy and reliability.

In the case of a continuous process, the values in the database represent samples of a constantly changing process variable. Any changes that occur in the variable between sample times will be lost. The decision on sampling frequency is a trade-off between more information (higher sampling rate) and compact data storage (lower sampling rate). Many process databases allow you to compress the data, as discussed earlier, to store more in a given amount of disk space.

Another critically important feature of a real-time process database is the ability to recover from computer and process upsets and continue to provide at least a basic set of process information to support a safe fallback operating condition, or else an orderly shutdown. A process plant does not have the luxury of taking hours or days to rebuild a corrupted database.

Most plants with real-time process databases archive the data as a history of past process operation. Recent data may be retained in disk storage in the plant's operating and control computers; older data may be written onto an offline disk drive or archival storage media such as CDs. With today's low costs for mass storage, there is little excuse not to retain process data for many years.

28.10 Data Quality Issues

Data quality is a matter of fitness for intended use. The data you need to prepare a water quality report for a governmental body will be different from the data required for fast-response control of a paper-machine wet end. In the broadest sense, data quality includes not only attributes of the numbers themselves, but how accessible, understandable and usable they are in their database environment. Figure 28-9 shows some of the dimensions, or aspects, of data quality.

Quality Category	Quality Dimensions
Intrinsic	Accuracy, Objectivity, Believability, Reputation
Accessibility	Access, Security
Contextual	Relevancy, Value-Added, Timeliness, Completeness, Amount of Data
Representational	Interpretability, Ease of understanding, Concise representation, Consistent representation

Figure 28-9: Aspects of Data Quality

Data from industrial plants is often of poor quality. Malfunctioning instruments or communication links may create ranges of missing values for a particular variable. Outliers (values which are grossly out-of-range) may result from transcription errors, communication glitches, or sensor malfunctions. An intermittently unreliable sensor or link may generate a data series with excessive noise variability. Data from within a closed control loop may reflect the impact of control actions rather than intrinsic process variability. Figure 28-10 illustrates some of the problems that may exist in process data. All these factors mean that data must often be extensively preprocessed before statistical or other analysis. In some cases, the worst data problems must be corrected and a second series of readings taken before analysis can begin.

28.11 Database Software

Many useful databases are built using off-the-shelf software such as MS Excel and MS Access. As long as query and report requirements are modest and real-time interaction with other computers or devices is not needed, this can be a viable and low-cost approach.

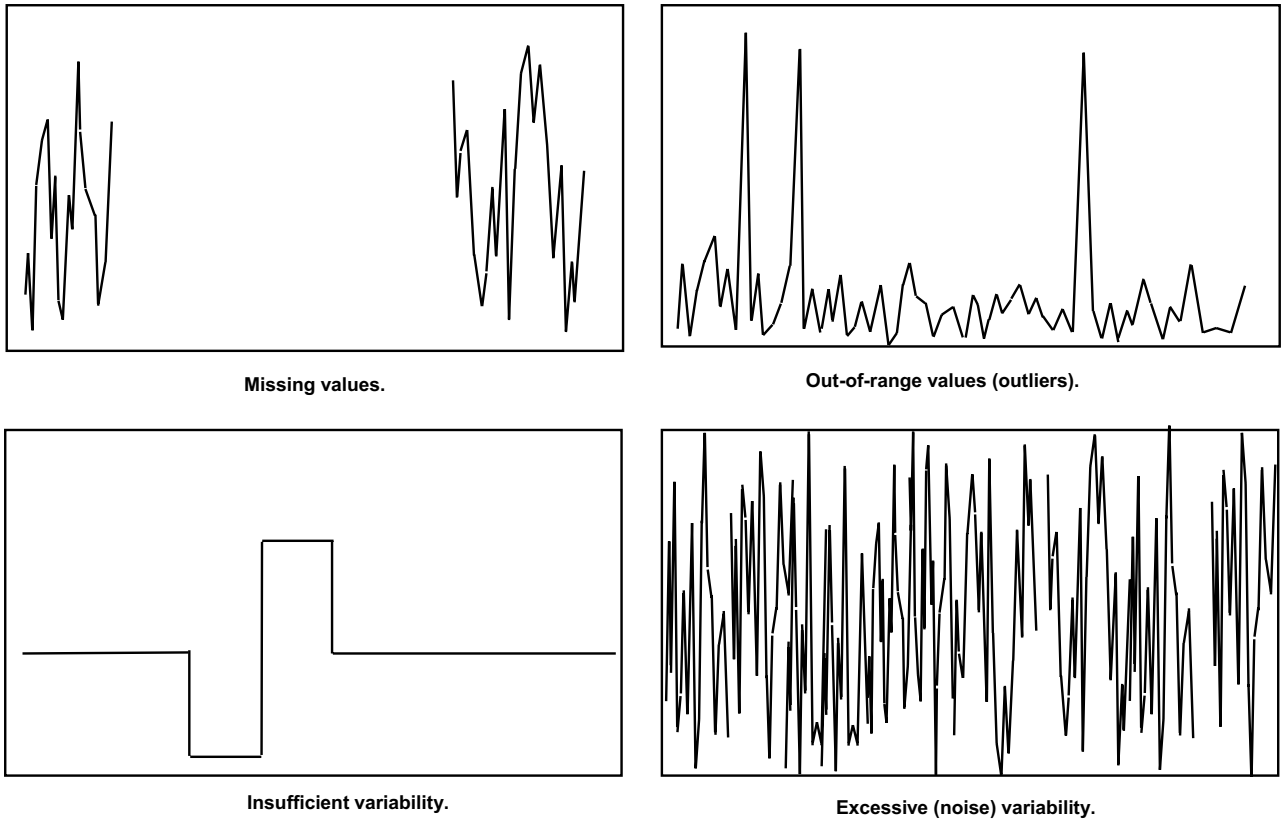


Figure 28-10: Common Problems with Process Data Quality

The next step up in sophistication is general-purpose business databases such are Oracle. If you choose a database that is a corporate standard, your database can work seamlessly with the rest of the enterprise data environment and use the full power of its query and reporting features.

However, business databases still do not provide many of the features required in a real-time process environment. A number of real-time process information system software packages exist, either general in scope or designed for particular industries. They may operate offline or else be fully integrated with the millwide process control and reporting system. Of course each level of sophistication tends to entail a corresponding increase in cost and complexity.

28.12 Data Documentation

Adequate data documentation is a frequently neglected part of database design. A database is a meaningless mass of numbers if its contents cannot be linked to the processes and products in your plant or office. Good documentation is especially important for numerical fields such as process variable values. At a minimum, the following information should be available: location and frequency of the measurement; tag number if available; how the value is obtained (sensor, lab test, panel readout, …); typical operating value and normal range; accuracy and reliability of the measurement; and any controllers whose operation may affect the measurement. Process time delays are useful information, since they allow you to lag values and detect correlations which include a time offset. A process diagram with measurement locations marked is also a helpful adjunct to the database.

28.13 Database Maintenance

Basic ongoing maintenance involves regular checks of the data for out-of-range data and other anomalies which may have crept in. Often the first warning of a sensor malfunction or dropout is a change in the characteristics of the data it generates. In addition, changing user needs are certain to result in a stream of requests for modifications to the database itself or to the reports and views it generates. A good understanding of database structure and functioning are needed to implement these changes while maintaining database integrity and fast, smooth data access.

Version upgrades in the database software pose an ongoing maintenance challenge. All queries and reports must be tested with the new version to make sure they still work, and any problems with users' hardware or software configurations or the interactions with other plant hardware and software must be detected and corrected. Additional training may be needed to enable users to benefit from new software features or understand a change in approach to some of their accustomed tasks.

28.14 Data Security

Data have become an indispensable resource for today's businesses and production plants. Like any other corporate asset, they are vulnerable to theft, corruption or destruction. The first line of defense is to educate users to view data as worthy of the same care and respect as other, more visible corporate assets. Protective measures such as passwords, firewalls, and physical isolation of the database servers and storage units are simply good practice. Software routines that could change access privileges, make major modifications to the database, or extract database contents to another medium must be accessible only to authorized individuals. Regular database backups, with at least one copy kept offsite, will minimize the loss of information and operating capability in case of an incident.

28.15 References

Date, C. J. *An Introduction to Database Systems*. Seventh Edition. Addison Wesley Longman, 1999.

Gray, J. "Evolution of Data Management." *IEEE Computer*, October 1999. pp. 38-46.

Harrington, J. L. *Relational Database Design Clearly Explained.* Second Edition. Morgan Kaufmann, 2002.

Litwin, P. *Fundamentals of Relational Database Design*. 2003. http://r937.com/relational.html.

Stankovic, J.A., S. H. Son, J. Hansson. "Misconceptions About Real-Time Data Bases." *IEEE Computer*, June 1999. pp. 29-36.

Strong, D. M., Y. W. Lee, R. Y. Wang. "Data Quality in Context." *Communications of the ACM.* Vol. 40, no. 5 (May 1997). pp. 103-110.

Wang, R. Y., V. C. Storey, C. P. Firth. "A Framework for Analysis of Data Quality Research." *IEEE Transactions on Knowledge and Data Engineering.* Vol. 7 (1995), no. 4. pp. 623-640.

About the Author

Diana C. Bouchard (Varanal Data Analysis) offers statistical data analysis services on a consulting basis, as well as scientific and technical writing, editing and translation. She holds an M.Sc. (Computer Science) degree from McGill University in Montreal and worked for 26 years as a scientist in the Process Control Group at the Pulp and Paper Research Institute of Canada (Paprican). Her activities at Paprican included modeling and simulation of kraft and newsprint mills, expert system development, and multivariate statistical data analysis. In the context of the Process Integration Chair at Ecole Polytechnique, she has lectured on steady state and dynamic simulation and multivariate data analysis.